Journal of Computer Applications ISSN 1001-9081
计算机应用 CODEN JYIIDU http://www.joca.cn

doi:10.11772/j.issn.1001-9081.yyyymmnnnn

面向高频 DApp 的 Hybrid TEE-Rollup 可恢复挑战协议与轻量 DA 成本评估

杨帆[1]

1. 华东师范大学 数据科学与工程学院，上海 200062

摘 要：高频去中心化应用（DApp）要求低延迟、低链上开销与公开可验证仲裁并存。本文面向 Hybrid TEE-Rollup 场景，提出可恢复挑战机制、携带证据的紧凑提交机制和数据可用性感知成本模型。可恢复挑战机制将超时会话恢复为可重放、可结算状态；紧凑提交绑定状态根、输出哈希、证明哈希、DA 指针与 DA 根；成本模型用于分析负载规模、批处理摊销与 DA 路径选择。基于 Python 原型、本地 EVM gas 测量与 Sepolia 部署记录，实验显示紧凑提交平均约 406 字节，在给定配置下模块化 DA 采样路径可降低约 96.33% 摊销成本，可恢复路径改善超时场景下的挑战完成能力。上述结论基于模拟 TEE、可验证 DA 注册表和配置化成本模型。

关键词：Hybrid TEE-Rollup；交互式挑战；数据可用性；高频 DApp；成本摊销；故障恢复

中图分类号:文献标志码：A 中图分类号：TP311

Recoverable Challenge Protocol and Lightweight DA Cost Evaluation for Hybrid TEE-Rollup in High-Frequency DApps

YANG Fan[1]

1. School of Data Science and Engineering, East China Normal University, Shanghai 200062, China

Abstract: High-frequency decentralized applications (DApps) require low latency, low on-chain overhead, and publicly verifiable arbitration. This paper proposes a recoverable challenge mechanism, an evidence-carrying compact commitment mechanism, and a data-availability-aware cost model for Hybrid TEE-Rollup. The recoverable challenge mechanism restores timeout sessions to replayable and settleable states; the compact commitment binds the state root, output hash, proof hash, DA pointer, and DA root; and the cost model analyzes payload size, batching amortization, and DA path selection. Based on a Python prototype, local EVM gas measurements, and Sepolia deployment records, the compact commitment averages about 406 bytes, the modular DA sampling path reduces amortized cost by about 96.33% under the given configuration, and the recovery path improves challenge completion under timeout scenarios. The conclusions are based on a simulated TEE, a verifiable DA registry, and a configurable cost model.

Key words: Hybrid TEE-Rollup; interactive challenge; data availability; high-frequency DApp; cost amortization; fault recovery

1 引言

高频去中心化应用的瓶颈来自低价值高频交互与异常可验证性的耦合。以链上 AI 推理服务为例，用户连续提交提示，执行者在可信执行环境中生成响应并向链上提交紧凑提交；完整提示、响应与证明保存在数据可用性层。正常路径需要低延迟和低链上开销，异常路径则必须支持公开仲裁：当响应与数据可用性负载不一致，或挑战在中途超时时，争议应继续推进到重放与结算。

现有路线分别覆盖了这一需求的不同侧面。全链上执行公开验证能力强，但高频负载、存储和执行成本直接上链；完全链下执行降低延迟和成本，却缺少用户可依赖的公开仲裁路径；纯 TEE 执行降低正常路径验证成本，但仍受硬件和宿主环境影响。Hybrid TEE-Rollup 将 TEE 快速路径与链上异常路径结合[1]，其风险也集中在异常路径：一旦恢复机制和证据入口缺失，系统会在最需要仲裁时失去协议连续性。

超时后挑战流程卡死属于协议活性与仲裁连续性问题。传统挑战机制可以关注错误是否能够被发现，但对于高频 DApp 而言，检测只是争议处理的起点：未完成的挑战会占用状态、延迟结算，并使错误提交停留在不可结算的中间状

态。因此，本文以故障发生、恢复、重放、仲裁结算为主线，将紧凑提交作为最小可验证证据入口，使正常路径保持轻量，异常路径仍可进入验证与重放。

实验部分采用合成负载、模拟 TEE、可验证 DA 注册表、本地 EVM gas 实测与 Sepolia 可部署性证据。

本文的贡献如下

（1）提出可恢复挑战协议。传统挑战机制无法处理交互超时问题，异常即便被检测也难以推进至重放与结算阶段。本文将会话恢复从单纯的工程补救手段，升级为保障协议活性的标准化状态转移机制。该机制在不篡改争议事实的前提下，将停滞的挑战会话切换至可重放、可仲裁的正常链路，从协议层面保障仲裁连续性。。

（2）设计证据绑定型紧凑提交机制。该结构将状态根、输出哈希、证明哈希、DA 指针与 DA 根进行密码学绑定，使链上精简凭证具备重放前置能力、证据可追溯能力与可验证属性。

（3）构建 DA 感知成本摊销模型。该模型量化分析负载规模伸缩、批处理策略、DA 部署路径三类因素对系统成本的影响，为不同业务负载下的 DA 方案选型与成本管控提供理论依据。

2 相关工作

TEEROLLUP 提出了利用异构 TEE 降低 Rollup 验证成本的系统设计。本文受其思路启发，但聚焦异常场景下的挑战恢复和紧凑提交的证据绑定。

opML 与 optimistic fraud proof 等工作研究如何通过乐观证明支持复杂计算，通常包含二分定位、单步仲裁和链上验证。本文借鉴交互式争议定位思想，但不实现完整 FPVM 或真实 ML 执行证明，而将二分重放用于 Hybrid TEE-Rollup 原型中的异常仲裁。

Dynamic Fraud Proof 关注无争议场景下的最终性压缩，并在发现争议时动态延迟结算。该方向强调挑战窗口和验证者参与机制的动态调整；本文聚焦更窄的超时后挑战会话恢复问题。

LazyLedger、模块化 DA 以及轻客户端相关工作强调数据可用性与执行验证解耦。本文使用可验证 DA 注册表和配置化建模分析紧凑提交加数据可用性验证的成本趋势。

Rollup 的基本思想是将大量执行过程移至链下完成，并将状态承诺、证明或争议入口提交到链上[2]。ZK-Rollup 通过有效性证明保证提交状态的正确性，验证属性较强，但证明生成、验证电路和系统实现成本较高。Optimistic Rollup 默认链下执行结果正确，仅在挑战期内出现有效争议时进行仲裁；其链上负担较轻，但争议窗口和交互过程会引入状态不确定性[3]。

乐观挑战是 Rollup 系统处理异常的重要机制。典型流程包括提交状态承诺、开启挑战、交互式缩小争议范围，并在必要时执行单步重放或链上仲裁[4][5]。对于长执行轨迹，二分定位能够将争议定位复杂度从线性扫描降低到对数轮次；挑战过程本身也可能遭遇超时、应答缺失或证据不可用，因此活性和恢复性需要进入协议设计[6]。

数据可用性关注链下或链外数据能否被验证者获取。模块化区块链和惰性账本等路线将数据可用性从执行验证中解耦，使共识层主要负责数据排序和可用性保证，执行与验证由独立的 Rollup 或客户端完成[7][8][9][10]。当执行者将完整负载放在数据可用性层而非链上时，链上提交必须与数据可用性条目形成可验证绑定。

在上述背景下，本文进一步限定研究问题：Hybrid TEE-Rollup 如何在保持正常路径低成本的同时，保留公开、可恢复且有证据支撑的异常路径。成本维度要求链上提交不随完整负载线性增长；可验证性维度要求紧凑提交能够指向可检查的 DA 证据；活性维度要求超时不会使仲裁流程永久停滞。

攻击面覆盖提交者、执行环境、数据可用性绑定和挑战流程。本文假设执行者或提交者可能提交错误响应、篡改数据可用性负载、提供无效证明，或在挑战流程中延迟响应；数据可用性层可能出现负载缺失、Merkle 证明损坏或根不一致；挑战流程本身也可能因交互超时而中断。本文不假设执行者在硬件 TEE 层面受到保护，也不假设数据可用性层具有无限可用性。

3 系统概览

系统包含三个角色，即执行层，DA 层和验证层：执行层负责产生正常路径下的响应与远程证明；DA 层负责保存重放所需证据；验证层负责将故障转化为挑战、恢复、重放与结算等状态转移。运行示例仍以高频 AI 推理工作负载为主：大量低价值请求不应迫使完整负载上链，但一旦发生争议，系统仍需保留公开仲裁所需的证据。

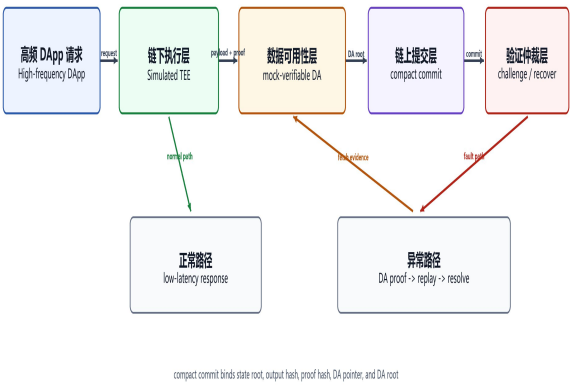


图 1 Hybrid TEE-Rollup 原型系统架构

Fig.1 Prototype architecture of Hybrid TEE-Rollup

如图 1 所示，系统由高频 DApp 请求入口、链下执行层、数据可用性层、链上提交层和验证仲裁层组成。正常路径中，执行层输出响应与远程证明，数据层保存完整证据，链上提交层仅保存紧凑提交。异常路径中，验证仲裁层通过数据可用性指针取回证据，并以挑战、恢复、重放和结算推进状态。这个划分使低成本正常路径与可仲裁异常路径保持同一证据上下文。

3.1 执行层与正常路径

执行层负责处理高频 DApp 请求。当前原型采用确定性模拟模型或可选模型后端生成响应，并使用模拟可信执行环境远程证明对输入哈希、输出哈希、nonce 和执行环境标识进行绑定[1]。远程证明的作用是为链下执行结果提供最小可信摘要，使后续验证层能够检查提交结果是否与声明的执行过程一致。

在本文原型中, 模拟可信执行环境用于建立“链下执行摘要—链上提交—异常挑战”的接口关系。

3.2 数据可用性接口

DA 接口是紧凑提交具有安全意义的关键位置。数据层保存完整负载及其可验证结构，包括提示、响应、远程证明、负载哈希、叶哈希、Merkle 根和字段级 Merkle 证明；链上仅保存紧凑提交。由此，字节压缩与证据保留被明确区分：正常路径避免完整负载上链，异常路径仍可恢复重放所需的负载上下文。

在运行示例中，如果执行者提交的响应被替换，验证者可以从数据可用性条目中取回响应字段及其 Merkle 证明，并与链上 DA Merkle 根检查一致性；如果负载缺失，则异常应被归类为数据可用性故障，而不应被误写为已完成重放的执行错误。

3.3 验证层与恢复触发

验证层负责处理异常路径。当前原型实现挑战打开、响应、步骤、恢复、重放和结算等状态推进；争议出现后，验证层通过证据加载、二分定位、单步重放和仲裁结算，将异常状态推进为可解释的协议结果[4][5]。

验证层同时承担故障分类职责。本文将故障划分为证明故障、DA 故障、超时故障、重放故障和轨迹不一致。不同故障对应不同处理路径：数据不可用表示数据层无法提供负载；数据可用性证明无效表示负载与链上根不一致；轨迹长度不匹配表示执行轨迹与声明结构不一致；响应篡改则可能同时导致负载哈希、远程证明和状态根不一致。

在 Solidity 映射中，原型包含 MockTEEVerifier、MockDARegistry、MerkleVerifier 和 HybridTEERollup 等合约，用于表达链上状态推进和最小验证逻辑。本地 EVM 合约并非 Python 原型的逐行移植，而是抽取适合链上表达的提交、验证、挑战、恢复和结算路径。

4 核心机制与成本评估

4.1 可恢复挑战协议

4.1.1 协议模型与形式化定义

本节给出恢复机制成立所需的协议边界。恢复操作只有在恢复流程推进而不修改争议事实时才具有安全意义。下面的定义将会话状态与挑战事实区分开，使超时后的恢复可以被理解为活性层转移，而不是隐藏的正确性裁决。

定义 1（挑战会话）

挑战会话定义为：$cs = (id, s, F, T)$。

其中，id 表示会话标识；s 表示当前状态；F 表示挑战事实（挑战事实集合）；T 表示超时参数。

定义 2（挑战状态机）

挑战状态机定义为：$M = (S, E, \delta)$。

其中，S 为状态集合；E 为事件集合；$\delta: S \times E \to S$ 为状态转移函数。

定义 3（恢复函数）

恢复函数定义为：Recover$(cs) = cs'$。

恢复仅改变活性相关状态，不改变挑战事实，即满足：$F(cs) = F(\text{Recover}(cs))$。

基于上述定义，恢复是活性层变换，而非正确性裁决。恢复函数允许会话从 TIMED_OUT 等停滞状态回到可推进状态，但不重写紧凑提交、数据可用性根、负载哈希、轨迹或不一致证据。该边界将恢复语义与罚没语义区分开，并说明恢复操作不会改变正确性: 后续重放仍基于原始挑战事实。

4.1.2 超时、恢复、重放与仲裁结算

可恢复挑战协议的目标，是将挑战从一次性错误检测流程扩展为异常场景下可恢复推进的协议状态机。其基本状态包括协议状态覆盖待处理、已打开、已响应、二分定位、已恢复、待重放、已重放、已结算、罚没和最终确认等阶段。

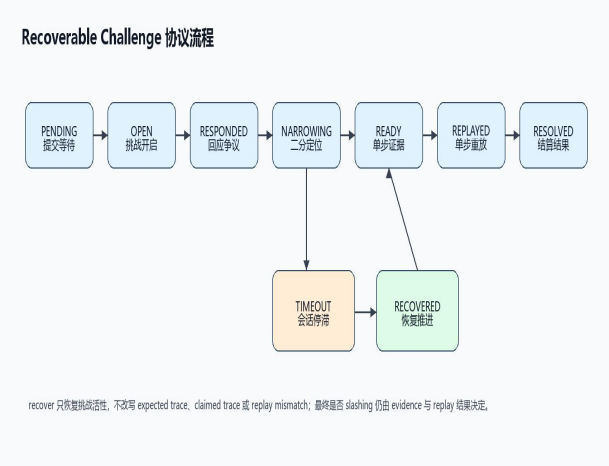


图 2 可恢复挑战协议流程

Fig.2 Workflow of the recoverable challenge protocol

当交易处于待定状态时，挑战者可以调用打开操作进入已打开状态。系统加载紧凑提交、数据可用性状态和相关不一致证据。随后，被挑战方进入响应阶段，提供其对争议的回应。若争议涉及执行轨迹，协议进入二分步骤阶段，通过二分缩小定位争议区间。对于长度为 n 的轨迹，理想情况下定位轮次接近 log2(n)，从而避免对长执行轨迹进行线性扫描。

当争议区间被缩小到单步时，协议进入单步重放。重放阶段比较预期哈希与声明哈希，判断局部执行是否一致。如果重放失败，结算阶段可以将交易标记为罚没状态；如果重放通过，则挑战不成立或进入相应结算逻辑。

恢复是协议中的关键状态转移，因为它在不改变证据内容的前提下恢复流程推进能力。在不可恢复的挑战中，超时可能使会话停留在“争议已知但无法重放和结算”的状态。可恢复挑战机制引入观察节点恢复路径，使挑战会话在异常场景下能够恢复推进，并重新回到可进行二分步骤、重放和结算的状态。这里需要区分活性与正确性：恢复操作恢复的是前者，而后续重放仍负责验证后者。

协议流程可分为三条路径。第一是正常路径：若挑战期内没有有效争议，交易最终进入最终确认。第二是可仲裁异常路径：若响应篡改或轨迹长度不匹配等执行相关故障出现，协议通过打开、响应、二分步骤、重放与结算完成争议定位和结算。第三是超时恢复路径：若挑战过程因超时中断，观察节点可以调用恢复将会话推进到已恢复状态，并继续后续重放与结算。该划分有助于避免将异常检测、活性恢复和经济惩罚混写为同一性质。

4.1.3 正确性边界与协议性质

性质一，数据可用性绑定性质。紧凑提交中的 DA Merkle 根与 DA 条目中的字段级 Merkle 证明之间建立了密码学绑定关系，因此提示词、响应或证明字段被替换后，会导致负载哈希、叶哈希或 Merkle 证明与链上根不一致。该性质不依赖真实 DA 网络，只说明当前可验证 DA 结构能够检测字段篡改，并保证紧凑提交与完整负载之间在证据层面不可脱钩。

性质二，恢复机制的仲裁连续性。恢复操作的目标是恢复超时挑战会话的流程推进能力，使其继续进入二分缩小、重放和结算；它不修改预期执行轨迹、声称执行轨迹或重放不一致的事实。因此，恢复改变的是挑战流程的活性连续性，而非争议正确性本身。该性质将恢复语义与安全裁决语义明确分开，避免将“流程可以继续”与“裁决结果改变”这两个不同层次的性质混淆。

性质三，二分定位性质。对于长度为 n 的执行轨迹，二分挑战步骤每轮缩小争议区间，理想轮次接近 log2(n)。本文实验中轨迹步数为 4、8、16、32、64、128 时，平均二分轮次分别为 2、3、4、5、6、7，与理论对数趋势一致。该性质说明争议定位的开销增长与理论对数趋势一致，使得执行轨迹较长时挑战的交互成本仍可接受。

4.1.4 协议性质

前述讨论已将可恢复挑战机制的核心性质从“超时后可以重试”这一工程事实提升为协议层面的状态转移机制。本节进一步将这一认识系统化为挑战事实不变性、安全性与活性的分离、仲裁连续性和恢复操作正确性四项性质，并给出协议不变量表格，以明确恢复机制的协议语义边界。

挑战事实不变性

挑战会话在创建时便将一组不可修改的协议事实绑定在会话上下文中，这些事实构成争议的“内容面”，包括：

- 状态根（State Root）：约束输入与输出哈希的绑定关系；
- 数据可用性根（DA Merkle Root）：约束 DA 层中完整负载的字段级承诺；
- 负载哈希（Payload Hash）：标识 DA 层中完整的原始负载；
- 执行轨迹（Execution Trace）：挑战方与被挑战方各自提交的执行步骤序列；
- 重放证据：二分缩小后定位到的目标步骤与对应的预期哈希和声明哈希；
- 不一致证据：验证、响应和 DA 字段检查中观测到的字段级不一致记录。

记挑战会话 cs 的挑战事实集合为 Facts(cs)，则恢复操作满足 $\mathrm{Facts}(cs_{\mathrm{before}}) = \mathrm{Facts}(cs_{\mathrm{after}})$，即恢复不向挑战会话注入新的状态根、负载哈希、执行轨迹或重放证据。

恢复操作恢复的是流程推进能力，而非修改争议内容本身。在不可恢复的挑战协议中，超时会使会话不可逆地停滞，已观测到的异常无法进入重放与仲裁。可恢复挑战机制将检测与处理之间的鸿沟打通，同时保证处理过程不脱离原始检测事实。

安全性与活性分离

恢复机制设计中的一个关键考量是安全性与活性的边界划分。恢复操作解决的是活性问题——当挑战会话因交互超时而停滞时，恢复使其重新获得推进能力；但它不直接改变正确性判断——一个原本错误的执行结果不会因恢复而变为正确。

从安全性角度看，恢复操作受到以下约束：第一，恢复后的重放阶段仍对预期哈希与声明哈希进行一致性检查，两者不一致时挑战成功；第二，恢复不修改重放证据中的目标步骤，二分缩小定位到的争议点在恢复前后保持不变；第三，恢复不修改负载承诺，紧凑提交中的 DA Merkle 根和 DA 指针在提交时已锁定。

综合来看，恢复机制恢复的是协议活性，而非重新定义正确性。正确性的最终裁决由重放阶段完成，恢复只保证该裁决环节能够被执行。

仲裁连续性

仲裁连续性用于刻画挑战协议活性性质的概念：在挑战事实已经固定、重放证据可获得且数据可用性证明可检查的条件下，若争议已经被检测到，则协议应保证该争议不会因交互超时永久停滞，而能够继续推进至重放、结算或惩罚阶段。本文将满足这一性质的协议称为具备仲裁连续性。

传统基于超时的挑战协议可以完成异常检测——验证者观测到 DA 不一致、响应不匹配或证明无效后，能够将异常记录为争议。然而，当挑战方或被挑战方在交互中超时未响应时，争议可能永久停留在中间状态，交易既不能最终确认也不能被罚没。这就是检测性与完成性的差异：传统方案关注“能否发现异常”，仲裁连续性关注“已发现的异常能否被处理完毕”。对于高频去中心化应用，未完成的挑战会话会占用状态、延迟结算，使链上对象长期处于不确定状态，直接损害结算确定性。

可恢复挑战机制通过恢复函数在超时后恢复仲裁流程推进能力：观察节点调用恢复功能使超时会话回到可推进状态，从而继续二分缩小、重放和结算。实验结果表明，在当前原型设定下，恢复机制能够将超时停滞转化为可继续推进的仲裁流程。

4.2 携带证据的紧凑提交机制

本节说明紧凑提交为什么不是单纯压缩结构。其研究价值在于携带异常路径所需的最小证据关联。该提交绑定状态根、输出哈希、证明哈希、数据可用性指针和数据可用性根：状态根约束输入、输出与环境摘要；输出哈希约束响应；证明哈希约束证明；数据可用性指针定位完整负载；数据可用性根则承诺字段级证明成员关系。

这种绑定关系使验证者能够从链上提交进入数据可用性证据结构。若响应被篡改，验证者检查响应叶与 Merkle 根；若证明与输出不一致，证明哈希和负载哈希形成交叉约束；若数据可用性证明损坏，系统可以将异常定位为证明无效，而不是混同为执行重放失败。由此，紧凑提交支持后续挑战和重放的证据加载，而不仅服务于链上字节压缩。

从对抗角度看，提交者可能替换负载字段、隐藏数据可用性条目、提交与证明不一致的输出，或在争议过程中延迟响应。紧凑提交的作用在于锁定链上最小承诺所需的证据字段，使后续挑战具备重放准备性，而不仅是节省链上字节。

4.3 数据可用性感知成本评估

成本模型关注全链上与紧凑提交配合数据可用性在高频场景下的结构性差异。全链上路线将完整负载作为链上数据提交，其成本随负载大小增大近似线性增长。紧凑提交配合数据可用性将负载发布从链上调用数据转移到数据可用性配置，链上只保留紧凑提交，从而在高频大批量场景中控制成本增长。

模型比较四类 DA 路径：全链上调用数据、紧凑外部 DA、紧凑类 EIP-4844 方案和紧凑模块化 DA 采样[12]。该比较作为研究原型的成本建模工具，用于刻画不同数据发布路径的结构性差异。

模型表达为 $C_{\text{total}} = C_{\text{commit}} + C_{\text{DA}} + p_{\text{challenge}} \times C_{\text{challenge}}$。其中 C_commit 表示链上紧凑提交或完整负载提交成本，C_DA 表示不同 DA 路径下的数据成本，C_challenge 表示争议发生时挑战、恢复、重放和结算成本，p_challenge 表示争议发生概率。批处理后单笔摊销成本记为 $C_{\text{amortized}} = \frac{C_{\text{total}}}{batch_{\text{size}}}$。

从系统角度看，批大小越大，固定提交成本越容易被摊薄；负载越大，全链上调用数据的线性成本越突出；数据可用性路径越接近低边际数据成本，紧凑提交配合数据可用性的收益越明显。因此，该模型的核心启示不在于某项数字，而在于批处理规模、负载大小和数据可用性路径选择三者之间存在结构性权衡。

5 实验评估

实验评估围绕机制可运行性与对比趋势展开。当前实验使用 Python 原型、模拟 TEE、合成负载、可验证 DA 注册表、本地 EVM gas 测量和 Sepolia 测试网部署记录。RQ1 关注紧凑提交是否在保留证据关联的同时隔离负载增长；RQ2 关注批处理摊销与 DA 路径选择的影响；RQ3 关注恢复机制是否改善超时场景下的挑战活性；RQ4 关注不同故障类型能否被检测、恢复、重放或罚没；RQ5 关注 Solidity 合约、本地 EVM 实测与 Sepolia 部署能否支撑链上对应物。

$$C_{\text{total}} = C_{\text{commit}} + C_{\text{DA}} + p_{\text{challenge}} \times C_{\text{challenge}} \quad (1)$$

$$C_{\text{amortized}} = \frac{C_{\text{total}}}{batch_{\text{size}}} \quad (2)$$

其中，C_commit 表示链上紧凑提交或完整负载提交成本，C_DA 表示不同 DA 路径下的数据成本，C_challenge

表示争议发生时挑战、恢复、重放和结算成本, p_challenge 表示争议发生概率。

5.1 紧凑提交能否降低链上提交规模

RQ1 隔离考察提交结构。Python 原型在模拟 TEE 下生成合成账本条目，比较完整负载与紧凑提交的规范 JSON 字节数。度量指标包括完整字节数、紧凑字节数和字节降幅比例。完整字节数表示完整提示、响应和远程证明都作为负载的一部分时的总字节数；紧凑字节数仅计算提交结构字节数。

对比结果呈现出清晰的隔离关系：紧凑提交基本保持在 406 字节左右，而完整负载字节数随负载大小增大而增长。当负载大小为 8192 时, 提示长度为 64 和 256 的完整负载字节数分别约为 50068.84 和 50262.84，而紧凑提交仍为 406.00 字节。这说明紧凑提交能够隔离负载规模扩展与链上提交规模，同时保留重放准备能力。

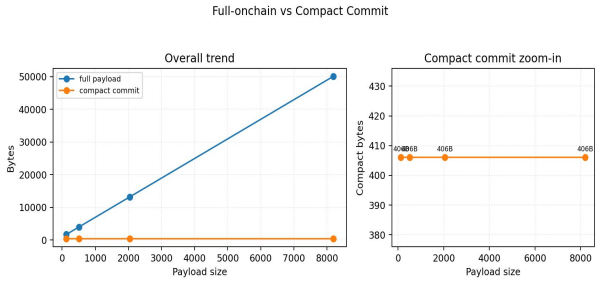


图 3 完整负载与紧凑提交的字节规模对比

Fig.3 Byte-size comparison between full payloads and compact commits

5.2 批大小如何影响 DA 感知摊销成本

RQ2 关注提交策略在何种条件下会改变高频合成负载的系统可行性。实验比较全链上调用数据、紧凑外部 DA、紧凑类 EIP-4844 方案和紧凑模块化 DA 采样四类路线。主要指标为摊销 gas 用量和降幅比例。

趋势表明，当负载发布成为主要开销时，批处理摊销的作用会更加明显。以负载 8192 为例，全链上调用数据的估算 gas 用量从批大小 1 时的 801101.44 下降到批大小 1000 时的 801.21；而紧凑模块化 DA 采样从 29026.98 降至 29.03。96.33% 是给定成本模型与数据可用性配置下的趋势结果, 说明紧凑模块化 DA 采样在大负载与大批处理条件下具有结构性优势。

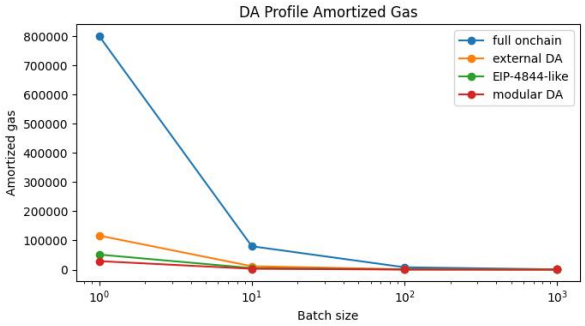


图 4 不同数据可用性配置下的摊销 gas 用量趋势

Fig.4 Amortized gas trends under different DA configurations

5.3 恢复机制能否改善超时场景下的挑战活性

RQ3 检验本文的核心活性主张。该实验对应第 4.1.4 节定义的仲裁连续性。Python 原型在不可恢复路径与可恢复路径两条路径中使用相同的合成挑战状态机，轨迹步数覆盖 4 至 128，挑战超时设为 2 秒。关键问题是：超时被检测后，会话是否仍能继续进入重放、结算与罚没。

二分定位行为与理论对数趋势一致。更关键的是不可恢复与可恢复路径的对比：在超时不可恢复场景中，检测率达到 100%，但挑战成功率和罚没比例均为 0%；在超时可恢复场景中，在当前原型合成负载下均可达到 100%。该对比说明，超时检测本身并不等价于协议推进。

利用表 5 中已有本地 EVM 数据，challengeStep 平均 gas 为 93807，challengeRecover 平均 gas 为 156891，恢复相对单轮挑战步骤的开销比约为 1.67。该成本发生在异常路径，对比对象应为未解决仲裁与有界恢复转移之间的差异，而非正常路径吞吐与恢复开销。

该比例仅表示本地 EVM 中两条合约路径的相对开销。

5.4 不同故障场景能否被检测、恢复或惩罚

RQ4 采用面向故障分类的安全评估方式。实验注入八类合成场景，区分其在当前原型中能否被检测、能否恢复推进、能否进入重放以及能否触发罚没。指标为检测率、挑战成功率和罚没比例。

正常路径无故障，检测率为 0%，直接进入最终确认。响应篡改和轨迹长度不匹配在当前原型中可检测、可恢复、可重放并可触发罚没，形成完整的挑战到罚没闭环。远程证明无效、数据不可用和数据可用性证明无效在当前原型中体现为检测和拒绝能力，尚未形成完整的挑战到罚没路径。超时不可恢复场景体现为检测但无法完成仲裁；超时可恢复场景则形成完整闭环。

表 4 失败场景分类与当前证据边界

Table 4 Failure scenarios and evidence boundaries

类别	场景	当前结果	证据边界
正常路径	正常路径	不触发检测或罚没	说明正常路径不误报
可检测但未统一惩罚	远程证明无效，数据不可用，数据可用性证明无效	检测率 100%，挑战成功率和罚没比例为 0%	说明可检测或拒绝，不说明已统一惩罚
挑战 + 惩罚闭环	响应篡改，轨迹长度不匹配	检测率、挑战成功率和罚没比例均为 100%	说明典型执行异常可进入闭环
超时活性对比	超时不可恢复 vs 超时可恢复	不可恢复路径为 0%，恢复后为 100%	说明恢复改善仲裁活性

5.5 Solidity、本地 EVM 与 Sepolia 证据能否支撑链上对应物

RQ5 评估链上合约对应物。当前项目已将核心路径映射为 Solidity 合约，并在 Hardhat 本地 EVM 测试链上得到 gas 用量基线，见表 5。同一套合约已部署至 Sepolia 测试网，Chain ID 为 11155111，部署区块为 10825904。这些结果说明原型路径具有合约对应物和公开测试网可部署性，不应被解释为生产可用性。

表 5 本地 EVM 关键路径平均 gasUsed

Table 5 Average gasUsed of local EVM key paths

操作	gasUsed	含义
registerDA	113526	MockDARegistry 记录负载哈希与数据可用性根
submitRollup	299033.67	紧凑提交提交路径
challengeOpen	283769	打开争议并加载证据
challengeRespond	65886	被挑战方响应
challengeStep	93807	二分定位的单轮推进
challengeRecover	156891	恢复超时会话
challengeReplay	135632	单步重放仲裁
challengeResolve	113402	结算争议结果
challengeFinalize	33477	挑战期后最终确认

实验的可复现性通过以下证据类型支撑：Python 原型脚本可生成完整实验输出；成本实验和挑战实验均使用确定性合成模型，不依赖外部推理服务；Hardhat 测试提供合约层面的可重复验证。所有实验输出文件（成本网格、挑战网格、故障场景、汇总文件）由同一实验脚本生成。

5.6 实验小结

上述 RQ1 至 RQ5 已经包含了部分对比分析。总体来看，可恢复路径使超时场景从检测停滞推进到可重放与结算；紧凑提交将链上提交稳定在约 406 字节；DA 感知成本模型说明批处理和数据可用性路径共同决定摊销成本；故障分类则区分检测、拒绝、重放与罚没等不同协议能力。

上述对比表明，本文机制的主要贡献在于组合轻量提交、证据绑定和可恢复仲裁路径，使高频场景下的异常处理能够继续推进。

6 结论

本文面向高频去中心化应用中的低延迟、低链上开销与公开可验证仲裁需求，提出了面向 Hybrid TEE-Rollup 的可恢复挑战机制、携带证据的紧凑提交机制以及数据可用性感知成本模型。可恢复挑战机制将超时会话恢复为可重放、可结算状态，强调恢复操作只恢复协议活性而不改变争议事实；紧凑提交机制绑定状态根、输出哈希、证明哈希、DA 指针与 DA 根，为异常路径提供最小证据入口；成本模型则用于分析负载规模、批处理摊销与 DA 路径选择之间的关系。

基于 Python 原型、本地 EVM gas 测量与 Sepolia 部署记录，实验结果表明，紧凑提交能够隔离链上提交规模与完整负载增长，在给定配置下模块化 DA 采样路径具有明显的摊销成本优势，恢复路径也能改善超时场景下的挑战完成能力。需要说明的是，当前结果基于模拟 TEE、可验证 DA 注册表和配置化成本模型，尚不代表真实主网或生产环境表现。后续工作将进一步接入真实 TEE 与数据可用性网络，并完善并发挑战、恢复频率限制和故障到罚没的统一处理机制。

参考文献参考文献：

[1] WEN X, FENG Q, LYU H, et al. TEEROLLUP: efficient Rollup design using heterogeneous TEE[EB/OL]. arXiv:2409.14647, 2024 [2026-06-15].
[2] BUTERIN V. An incomplete guide to Rollups[EB/OL]. 2021 [2026-06-15].
[3] ETHEREUM FOUNDATION. Optimistic Rollups[EB/OL]. 2024 [2026-06-15].
[4] ARBITRUM FOUNDATION. Arbitrum Nitro technical documentation[EB/OL]. 2024 [2026-06-15].

[5] OP LABS. Optimism documentation: fault proofs and dispute games[EB/OL]. 2024 [2026-06-15].
[6] PICCO G, FORTUGNO A. Dynamic fraud proof[EB/OL]. arXiv:2502.10321, 2025 [2026-06-15].
[7] AL-BASSAM M. LazyLedger: a distributed data availability ledger with client-side smart contracts[EB/OL]. arXiv:1905.09274, 2019 [2026-06-15].
[8] TAS E N, TSE D, YANG L, et al. Light clients for lazy blockchains[EB/OL]. arXiv:2203.15968, 2022 [2026-06-15].
[9] EIGENLABS. EigenDA documentation[EB/OL]. 2024 [2026-06-15].
[10] CELESTIA. Celestia: modular blockchain network[EB/OL]. 2024 [2026-06-15].
[11] INTEL CORPORATION. Intel Software Guard Extensions developer guide[EB/OL]. 2024 [2026-06-15].
[12] ETHEREUM FOUNDATION. EIP-4844: shard blob transactions[EB/OL]. 2024 [2026-06-15].

联系人：杨帆
通信地址：上海市普陀区中山北路 3663 号华东师范大学数据科学与工程学院，200062
电子信箱：2991777340@qq.com
电话：13319693336